IF3170 Inteligensi Artifisial

LAPORAN TUGAS BESAR 1

Disusun oleh:

Nathan E C Marpaung — 13524062

Kurt Mikhael Purba — 13524065

Aurelia Jennifer Gunawan — 13524089

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

1 Deskripsi Persoalan **1**

2 Dasar Teori **3**
2.1 Algoritma *Hill Climbing* . . . 3
2.1.1 *Steepest Ascent Hill Climbing* . . . 3
2.1.2 *Hill Climbing with Sideways Move* . . . 4
2.1.3 *Random Restart Hill Climbing* . . . 5
2.1.4 *Stochastic Hill Climbing* . . . 7
2.2 Algoritma *Simulated Annealing* . . . 8
2.3 Algoritma *Genetic Algorithm* . . . 10

3 Implementasi **12**
3.1 Arsitektur dan Struktur Program . . . 12
3.2 Representasi State, Fungsi Objektif, dan Constraints . . . 14
3.2.1 Representasi State . . . 14
3.2.2 Fungsi Objektif . . . 15
3.2.3 Constraints . . . 15
3.3 Representasi Neighbor dan State Awal . . . 19
3.3.1 Pembangkitan State Awal . . . 19
3.3.2 Operasi Tetangga . . . 20
3.4 Implementasi Kelas-kelas Pendukung . . . 24
3.4.1 truck.py . . . 24
3.4.2 package.py . . . 25
3.4.3 problem.py . . . 27
3.4.4 placement.py . . . 27
3.4.5 state.py . . . 28
3.4.6 reader.py . . . 30
3.4.7 common.py . . . 31
3.5 Implementasi Algoritma Hill-climbing . . . 33
3.5.1 hill_climb.py . . . 33
3.6 Implementasi Algoritma Simulated Annealing . . . 41
3.6.1 simulated_annealing.py . . . 41
3.7 Implementasi Algoritma Genetic Algorithm . . . 48
3.7.1 genetic_algorithm.py . . . 48

4 Hasil Eksperimen dan Analisis **60**
4.1 Eksperimen Algoritma *Hill-climbing* . . . 60
4.1.1 *Steepest Ascent Hill-climbing* . . . 60
4.1.2 *Hill-climbing with Sideways Move* . . . 60
4.1.3 *Random Restart Hill-climbing* . . . 60
4.1.4 *Stochastic Hill-climbing* . . . 60
4.2 Eksperimen Algoritma *Simulated Annealing* . . . 60
4.3 Eksperimen Algoritma *Genetic Algorithm* . . . 60
4.4 Analisis Hasil Eksperimen . . . 60

5 Kesimpulan dan Saran **61**
5.1 Kesimpulan . . . 61
5.2 Saran . . . 61

Referensi **62**

Lampiran **63**
Pembagian Tugas . . . 63
Form Pengunaan AI . . . 64

Bab 1

Deskripsi Persoalan

Pemuatan barang ke dalam kendaraan angkut merupakan salah satu permasalahan umum dalam industri logistik yang melibatkan berbagai variabel, seperti dimensi paket, berat, nilai barang, kerapuhan, keterbatasan ruang, dan kapasitas kendaraan. Penentuan paket yang dimuat dan penempatannya dalam peti kemas truk memerlukan pengaturan yang efisien agar tidak terjadi pelanggaran, seperti paket yang melayang atau menembus dinding peti kemas, barang pecah-belah yang tertindih paket lain, atau total berat muatan yang melebihi kapasitas truk. Apabila dilakukan secara manual, proses ini sangat tidak efektif karena ruang solusi yang meningkat secara eksponensial seiring bertambahnya jumlah paket, dengan kombinasi pilihan paket, posisi koordinat, dan enam kemungkinan orientasi untuk setiap paket.

Tugas besar 1 mata kuliah IF3170 Inteligensi Artifisial bertujuan agar peserta kuliah memahami dan mampu mengimplementasikan serta mengevaluasi berbagai algoritma *local search* untuk menyelesaikan permasalahan pengisian paket pada truk peti kemas Mejiro Express. Persoalan ini merupakan variasi dari *3D bin packing* dan *knapsack problem*, dengan tujuan memaksimalkan total nilai paket yang berada dalam peti kemas. Tidak seluruh paket harus dimuat, sehingga paket yang tidak terpilih dapat ditempatkan di luar truk. Dalam konteks inteligensi artifisial, proses ini dapat dimodelkan sebagai sebuah *problem-solving agent* yang bertindak secara rasional dengan mempertimbangkan keterbatasan ruang dan kapasitas berat. Agen diharapkan dapat memilik aksi terbaik berdasarkan *state* saat ini untuk mencapai kondisi optimal.

Untuk mencapai tujuan tersebut, digunakan pendekatan *local search*. *Local search* cocok untuk masalah yang tidak membutuhkan pencarian jalur menuju solusi dan hanya berorientasi pada pencarian *final state* terbaik. Pencarian dimulai dari *initial state* yang dibangkitkan secara acak, kemudian dieksplorasi terhadap *state* tetangga untuk memperoleh nilai *objective function* yang lebih baik. Perpindahan antar-*state* dibatasi pada tiga jenis langkah, yaitu menukar dua paket, memindahkan sebuah paket ke koordinat yang berbeda, dan memutar paket 90 derajat terhadap sumbu x, y, atau z. Setiap *state* harus memenuhi batasan yang ditetapkan, yaitu paket tidak boleh melayang atau menembus dinding peti kemas, barang pecah-belah tidak boleh menjadi penyangga paket lain, dan total berat muatan tidak boleh melebihi kapasitas truk.

Dalam tugas besar 1, dilakukan implementasi beberapa varian algoritma *local search*, yaitu *Hill-climbing, Simulated Annealing*, dan *Genetic Algorithm*. Setiap algoritma memiliki karakteristik dan strategi pencarian yang berbeda. *Hill-climbing* berfokus pad aperbaikan bertahap menuju solusi terbaik, *Simulated-Annealing* mengombinasikan *Hill-climbing* dengan probabilitas berpindah ke solusi yang lebih buruk untuk menghindari *local optimum*, serta *Genetic Algorithm* yang mempertahankan sekumpulan solusi dan memperbaruinya melalui proses seleksi, *crossover*, dan mutasi. Hasil dari setiap algoritma kemudian dianalisis berdasarlan nilai *objective function* akhir, durasi pencarian, dan konsistensi hasil dari beberapa kali percobaan.

Bab 2

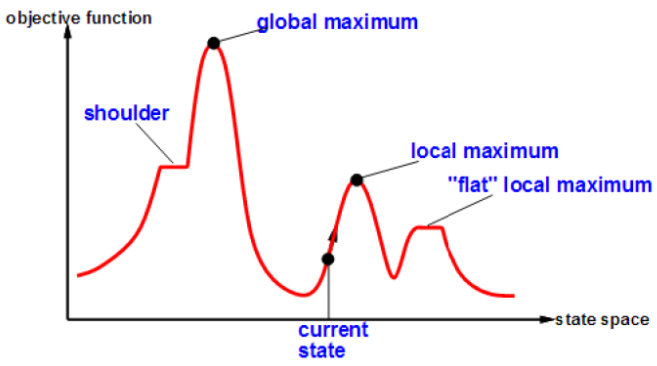
Dasar Teori

Local search merupakan pendekatan dalam kecerdasan buatan yang mencari solusi dengan berpindah dari satu *state* ke *state* tetangganya. *Classical search*, seperti BFS dan DFS, mementingkan jalur menuju solusi. Sedangkan *local search* hanya mempertahankan satu *state* aktif sehingga cocok untuk masalah yang hasil akhirnya lebih penting daripada urutan langkahnya. Setiap *state* merepresentasikan konfigurasi solusi yang lengkap. Perpindahan dilakukan dengan menghasilkan *successor* atau *neighbor* dari *state* saat ini, lalu kualitasnya dinilai menggunakan fungsi objektif. Algoritma akan memaksimalkan fungsi objektif atau meminimalkan fungsi biaya. Tujuan utama dari *local search* adalah untuk menemukan *global optimum*. Namun, pencarian dapat terhenti atau terjebak pada *local optimum*, yaitu *state* yang lebih baik daripada semua tetangganya tetapi bukan yang terbaik secara keseluruhan. Untuk mengatasi permasalahan ini, sebagian algoritma menambahkan mekanisme seperti langkah acak, penerimaan solusi yang sementara lebih buruk, atau pengembangan banyak kandidat sekaligus.

2.1 Algoritma *Hill Climbing*

2.1.1 *Steepest Ascent Hill Climbing*

Steepest ascent hill-climbing adalah varian dasar algoritma pencarian *hill-climbing*, yaitu algoritma pencarian lokal yang bergerak secara terus-menerus ke arah nilai yang lebih baik, baik berupa kenaikan nilai objektif maupun penurunan nilai biaya (*cost*).



Gambar 2.1: Grafik Fungsi Objektif terhadap State Space pada Steepest Ascent HC

Algoritma diawali dari sebuah *initial state* yang dibangkitkan secara acak. Pada setiap iterasi, algoritma membangkitkan seluruh *successor* dari *state* saat ini, lalu memilih *neighbor* yang merupakan *highest-valued successor*, yaitu *successor* dengan nilai tertinggi. Jika terdapat lebih dari satu *successor* terbaik, pemilihan dilakukan secara acak di antara *successor* tersebut. Berikut adalah pseudocode algoritma *steepest ascent hill-climbing*:

Algorithm 1: Hill-Climbing (Steepest Ascent)

Input: *problem*
Output: state yang merupakan local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
while true do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then
        return current.STATE
    current ← neighbor
```

Alur kerja algoritma *steepest ascent HC* terdiri dari tiga bagian. Pertama, inisialisasi dengan *state* acak. Kedua, *loop* yang terus berpindah ke *neighbor* terbaik selama nilainya lebih tinggi daripada *state* saat ini. Ketiga, terminasi ketika tidak ada *neighbor* yang memiliki nilai lebih tinggi dari *state* saat ini. Kondisi ini mencakup puncak maupun area datar, karena kondisi berhenti menggunakan operator $\leq$.

2.1.2 *Hill Climbing with Sideways Move*

Hill-climbing with sideways move adalah salah satu varian dari *steepest ascent hill-climbing* yang dirancang untuk mengatasi kegagalan algoritma dasar pada kondisi *shoulder*. Pada *steepest ascent HC*, pencarian berhenti ketika tidak ada *neighbor* yang bernilai lebih tinggi dari *state* saat ini. Kondisi berhenti ini juga terpicu pada area datar, padahal pada area tersebut, seperti *shoulder*, masih terdapat kemungkinan untuk mencapai nilai yang lebih baik jika pencarian dilanjutkan.

Algoritma HC dengan *sideways move* ini mengizinkan algoritma berpindah ke *neighbor* yang nilainya sama dengan *state* saat ini, yang disebut *sideways move*. Perubahan ini diwujudkan dengan mengganti kondisi terminasi dari $neighbor.Value \leq current.Value$ menjadi $neighbor.Value < current.Value$. Dengan perubahan tersebut, algoritma hanya berhenti apabila seluruh *neighbor* memiliki nilai yang benar-benar lebih rendah, sehingga perpindahan ke *neighbor* bernilai sama tetap diperbolehkan.

Perpindahan sideways tanpa batas dapat menyebabkan *infinite loop*, yaitu ketika algoritma terus berpindah antar-*state* bernilai sama pada *flat local maximum* yang bukan *shoulder*. Maka, jumlah *sideways move* yang dilakukan secara berurutan perlu dibatasi dengan suatu nilai batas. Apabila batas tersebut tercapai, algoritma dihentikan dan mengembalikan *state* saat ini. Nilai limit menjadi parameter yang memengaruhi keseimbangan antara peluang keberhasilan dan waktu komputasi. Berikut adalah pseudocode algoritma *hill-climbing with sideways move*:

Algorithm 2: Hill-Climbing dengan Sideways Move

Input: *problem*, *maxSideways*

Output: state yang merupakan local maximum

```
current ← MAKE-NODE(problem.INITIAL-STATE)
sideways ← 0
while true do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE < current.VALUE then
        return current.STATE
    if neighbor.VALUE = current.VALUE then
        sideways ← sideways + 1
        if sideways > maxSideways then
            return current.STATE
    else
        sideways ← 0
    current ← neighbor
```

2.1.3 *Random Restart Hill Climbing*

Random restart hill-climbing adalah salah satu varian *hill-climbing* yang mengatasi masalah terjebak pada optimum lokal dengan cara mengulang pencarian dari *state* awal yang berbeda. Algoritma ini menjalankan serangkaian *hill-climbing*, dan setiap pencarian dimulai dari *initial state* yang dibangkitkan secara acak. Proses diulang hingga ditemukan *state* yang memenuhi tujuan, yaitu *state* dengan nilai *global maximum*. Jika pencarian dimulai dari area yang mengarah ke *local maximum*, algoritma akan berhenti di sana dan tidak dapat keluar. Dengan memulai ulang dari *state* acak yang berbeda, setiap percobaan memiliki peluang tersendiri untuk berada pada area yang mengarah ke *global maximum*. Semakin banyak percobaan dilakukan, semakin besar peluang setidaknya satu di antaranya berhasil. Setiap percobaan merupakan

hill-climbing biasa dan bersifat independen terhadap percobaan sebelumnya.

Kinerja algoritma ini dapat dianalisis dengan probabilitas. Misalkan p adalah probabilitas sebuah percobaan hill-climbing berhasil mencapai solusi, s adalah rata-rata jumlah langkah pada percobaan yang berhasil, dan f adalah rata-rata jumlah langkah pada percobaan yang gagal. Maka jumlah *restart* yang diharapkan hingga ditemukan solusi adalah

$$E[\text{percobaan}] = \frac{1}{p} \tag{2.1}$$

dan jumlah langkah yang diharapkan adalah

$$E[\text{langkah}] = s + \frac{f(1-p)}{p} \tag{2.2}$$

Persamaan kedua diperoleh dari dua komponen. Rata-rata terdapat $\frac{1-p}{p}$ percobaan gagal sebelum percobaan pertama yang berhasil, dan tiap percobaan gagal membutuhkan f langkah. Percobaan terakhir yang berhasil menambah s langkah. Dari persamaan tersebut terlihat bahwa nilai p yang kecil membuat jumlah restart membesar, tetapi total langkah tetap dapat rendah selama setiap percobaan berlangsung singkat. Hill-climbing yang cepat berhenti sehingga cocok digunakan secara berulang.

Algoritma ini juga bersifat *complete* secara probabilistik. Selama $p > 0$, solusi pada akhirnya akan ditemukan dengan probabilitas mendekati 1 jika jumlah restart tidak dibatasi. Namun, waktu komputasi tidak dapat dipastikan karena bergantung pada keberuntungan pembangkitan state awal. Berikut adalah pseudocode algoritma *random restart hill climbing*

Algorithm 3: Random Restart Hill-Climbing

Input: *problem*

Output: state dengan nilai global maximum

```
while true do
    current ← MAKE-NODE(RANDOM-STATE(problem))
    while true do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then
            break
        current ← neighbor
    if current.VALUE = problem.GLOBAL-MAXIMUM then
        return current.STATE
```

2.1.4 *Stochastic Hill Climbing*

Stochastic hill-climbing adalah salah satu varian *hill-climbing* yang tidak mengevaluasi seluruh *successor* dari *state* saat ini. Pada *steepest ascent HC*, algoritma membangkitkan seluruh *successor* lalu memilih yang bernilai tertinggi. Pada varian ini, algoritma hanya membangkitkan satu *successor* secara acak, kemudian memutuskan apakah perpindahan dilakukan berdasarkan perbandingan nilainya dengan *state* saat ini. Algoritma akan berpindah ke *neighbor* acak tersebut hanya jika nilainya lebih baik daripada *state* saat ini. Jika *neighbor* yang terpilih lebih buruk atau sama, *neighbor* tersebut diabaikan dan algoritma mencoba membangkitkan *successor* acak yang lain.

Algoritma tidak dapat menentukan bahwa ia telah mencapai puncak hanya dari satu kali percobaan yang gagal, karena tidak ada jaminan bahwa *successor* acak yang dipilih lebih baik. Oleh karena itu, kondisi terminasi adalah tercapainya jumlah iterasi maksimum (n_{max}). Algoritma berhenti setelah n_{max} iterasi dan mengembalikan *state* terakhir yang dicapai. Dibandingkan dengan *steepest ascent*, *stochastic hill-climbing* memerlukan biaya per iterasi yang lebih rendah karena hanya satu *successor* yang dievaluasi, tetapi secara keseluruhan membutuhkan lebih banyak langkah untuk mencapai hasil yang sebanding sehingga berjalan lebih lambat. Algoritma ini tetap dapat terjebak pada *local maximum*, karena hanya menerima perpindahan yang memperbaiki nilai. Berikut adalah pseudocode algoritma *stochastic hill-climbing*:

Algorithm 4: Stochastic Hill-Climbing

Input: *problem*, n_{max}

Output: state hasil pencarian

```
current ← MAKE-NODE(problem.INITIAL-STATE)
for i ← 1 to n_max do
    neighbor ← a random successor of current
    if neighbor.VALUE > current.VALUE then
        current ← neighbor
return current.STATE
```

2.2 Algoritma *Simulated Annealing*

Simulated annealing adalah algoritma *local search* berupa versi *stochastic hill-climbing* yang mengizinkan sebagian langkah buruk agar dapat keluar dari *local maximum*. Frekuensi langkah buruk ini dikurangi secara bertahap selama pencarian berlangsung. Nama algoritma ini berasal dari proses *annealing* pada metalurgi, yaitu pemanasan logam lalu pendinginan perlahan. Analoginya, parameter T berperan sebagai "temperatur" yang bernilai tinggi di awal dan menurun mengikuti jadwal pendinginan, yaitu fungsi yang memetakan waktu t ke nilai T.

Pada setiap iterasi, algoritma memilih satu successor acak dari *state* saat ini, lalu menghitung selisih nilainya, yaitu $\Delta E = next.\text{VALUE} - current.\text{VALUE}$. Jika $\Delta E > 0$, *neighbor* lebih baik dan algoritma selalu berpindah. Jika $\Delta E < 0$, *neighbor* lebih buruk, tetapi algoritma tetap dapat berpindah dengan probabilitas $e^{\Delta E/T}$. Probabilitas ini makin kecil ketika langkah semakin buruk dan ketika T semakin rendah. Akibatnya, pada T tinggi algoritma berperilaku seperti *random walk*, sedangkan pada T mendekati 0 algoritma berperilaku seperti *stochastic hill-climbing*. Algoritma berhenti ketika $T = 0$ dan mengembalikan *state* saat ini.

$$\Delta E = next.\text{VALUE} - current.\text{VALUE} \tag{2.3}$$

Keputusan perpindahan bergantung pada tanda ΔE. Jika $\Delta E > 0$, *neighbor* lebih baik dan algoritma selalu berpindah. Jika $\Delta E < 0$, *neighbor* lebih buruk, tetapi algoritma tetap dapat berpindah dengan probabilitas

$$P = e^{\Delta E/T} \tag{2.4}$$

Karena $\Delta E < 0$, nilai P selalu berada di antara 0 dan 1. Probabilitas ini menurun ketika langkah semakin buruk (nilai $|\Delta E|$ makin besar) dan ketika temperatur semakin rendah. Pada temperatur tinggi, P mendekati 1 sehingga hampir semua langkah buruk diterima. Pada temperatur mendekati 0, P mendekati 0 sehingga langkah buruk hampir tidak pernah diterima.

Untuk menentukan apakah langkah buruk diterima, nilai P dibandingkan dengan suatu nilai pembanding. Terdapat dua pendekatan, yaitu membandingkan dengan nilai statis (misalnya langkah diterima jika $P > 0{,}5$), atau membandingkan dengan bilangan acak pada rentang $[0, 1]$ (langkah diterima jika $P > \text{random}(0, 1)$). Pendekatan kedua lebih sesuai dengan sifat stokastik algoritma karena langkah buruk yang kecil tetap memiliki peluang diterima secara proporsional dengan nilai P.

Algoritma berhenti ketika $T = 0$ dan mengembalikan *state* saat ini. Secara teoritis, apabila T diturunkan cukup lambat, *simulated annealing* akan menemukan global optimum dengan probabilitas mendekati 1. Kelemahannya, penurunan temperatur yang lambat membuat waktu pencarian lebih lama, sehingga pemilihan jadwal pendinginan menjadi parameter penting. Berikut adalah pseudocode algoritma *simulated annealing*:

Algorithm 5: Simulated Annealing

Input: *problem*, *schedule*
Output: state solusi

```
current ← MAKE-NODE(problem.INITIAL-STATE)
for t ← 1 to ∞ do
    T ← schedule(t)
    if T = 0 then
        return current.STATE
    next ← a randomly selected successor of current
    ΔE ← next.VALUE − current.VALUE
    if ΔE > 0 then
        current ← next
    else
        current ← next only with probability e^(ΔE/T)
```

2.3 Algoritma *Genetic Algorithm*

Genetic algorithm (GA) adalah algoritma *local search* yang mempertahankan sekumpulan *state* (populasi). Genetic algorithm membangkitkan *successor* melalui penggabungan dua *state* induk (*parent*). Setiap *state*, yang disebut individu, direpresentasikan sebagai *string* pada alfabet berhingga, umumnya *string* bernilai 0 dan 1. Setiap karakter pada *string* merepresentasikan satu variabel permasalahan. Kualitas setiap individu dinilai oleh fungsi evaluasi yang disebut fungsi kebugaran (*fitness function*), dengan nilai yang lebih tinggi menunjukkan *state* yang lebih baik. Fungsi *fitness* harus dirancang sesuai permasalahan, dan nilai maksimumnya menjadi global maximum.

Algoritma diawali dengan populasi awal berisi k individu yang dibangkitkan secara acak. Generasi berikutnya dibentuk melalui tiga operasi berikut.

1. **Selection**

   Dua individu induk dipilih secara acak dengan probabilitas proporsional terhadap nilai *fitness*. Probabilitas terpilihnya individu i adalah

   $$P(i) = \frac{F(i)}{\sum_{j=1}^{k} F(j)} \tag{2.5}$$

   Individu dengan *fitness* lebih tinggi lebih berpeluang terpilih, tetapi individu dengan *fitness* rendah tetap memiliki peluang. Implementasinya dapat berupa *roulette wheel*, yaitu membangkitkan bilangan acak pada rentang $[0, 100]$ lalu mencocokkannya dengan interval kumulatif probabilitas setiap individu.

2. **Crossover**

   Sebuah titik potong (*crossover point*) dipilih secara acak pada string. Anak dibentuk dengan menggabungkan bagian awal *string* induk pertama dan bagian akhir *string* induk kedua. Jika dilakukan pada kedua arah, dihasilkan dua anak.

3. **Mutation**

   Dengan probabilitas kecil, satu karakter pada *string* anak diubah secara acak pada titik mutasi (*mutation point*). Mutasi menjaga keragaman populasi dan memungkinkan munculnya karakteristik yang tidak dimiliki kedua induk.

Proses selection, crossover, dan mutation diulang hingga terbentuk populasi baru

berukuran sama dengan populasi sebelumnya, kemudian populasi lama digantikan. Iterasi berlanjut hingga ada individu yang cukup fit atau waktu yang ditentukan habis. Algoritma mengembalikan individu terbaik pada populasi menurut fungsi *fitness*. Berikut adalah pseudocode algoritma *genetic algorithm*:

Algorithm 6: Genetic Algorithm

Input: *population*, FITNESS-FN

Output: individu terbaik

```
repeat
    new_population ← ∅
    for i ← 1 to SIZE(population) do
        x ← RANDOM-SELECTION(population, FITNESS-FN)
        y ← RANDOM-SELECTION(population, FITNESS-FN)
        child ← REPRODUCE(x, y)
        if probabilitas acak kecil terpenuhi then
            child ← MUTATE(child)
        tambahkan child ke new_population
    population ← new_population
until ada individu yang cukup fit atau waktu habis
return individu terbaik pada population menurut FITNESS-FN
```

Bab 3

Implementasi

3.1 Arsitektur dan Struktur Program

Program pencarian konfigurasi pemuatan paket ke dalam peti kemas truk ini dibangun menggunakan bahasa pemrograman Python dengan pendekatan berorientasi objek. Entitas masalah, representasi solusi (*placement* dan *state*), dan ketiga algoritma *local search* dipisahkan ke dalam modul masing-masing. Pemisahan ini memungkinkan seluruh algoritma memakai fungsi pembangkit *state* awal, pembangkit tetangga, validasi *constraint*, dan *objective function* yang sama, sehingga perbandingan antar algoritma dilakukan pada kondisi yang setara. Perhitungan inti hanya memakai pustaka standar Python (json, random, math, time, dan argparse); pustaka pandas dan plotly hanya dipakai oleh modul visualisasi.

Tabel 3.1: Modul program

Modul	Deskripsi
main.py	Program utamayang membaca berkas masukan, memilih algoritma dan parameternya, menjalankan setiap algoritma beberapa kali dengan *seed* berbeda, lalu menyimpan hasil tiap *run* sebagai berkas JSON di output/<nama tc>/.
reader.py	Membaca berkas JSON masukan dan membentuk objek Problem yang berisi Truck dan daftar Package.
truck.py, package.py, problem.py	Merepresentasikan entitas masalah: dimensi dan kapasitas berat truk, atribut paket (dimensi, *value*, berat, status *fragile*, ETA), serta kumpulan keduanya. package.py juga menyimpan daftar enam orientasi paket.

Modul	Deskripsi
placement.py, state.py	Placement menyimpan posisi dan orientasi satu paket, sedangkan State menyimpan Placement seluruh paket.
constraints.py	Memvalidasi sebuah *state* terhadap seluruh *constraint* (dinding, *overlap*, kapasitas berat, paket melayang, dan paket *fragile*).
objective.py	Menghitung *objective function*, yaitu total *value* paket yang berada di dalam truk.
initialization.py	Membangkitkan *state* awal acak yang valid untuk *Hill-climbing* dan *Simulated Annealing*.
neighbors.py	Membangkitkan tetangga valid dari sebuah *state* dengan operator *move*, *rotate*, dan *swap*, baik secara menyeluruh maupun secara acak.
common.py	Menyusun format hasil *run* yang seragam untuk semua algoritma.
hill_climb.py	Implementasi *Hill-climbing* dengan empat varian: *steepest ascent*, *sideways move*, *random restart*, dan *stochastic*.
simulated_annealing.py	Implementasi *Simulated Annealing* dengan jadwal pendinginan geometris dan pencatatan kejadian *stuck*.
genetic_algorithm.py	Implementasi *Genetic Algorithm*, yaitu pencarian berbasis populasi dengan *roulette wheel selection*, *crossover*, dan *mutation*.
visualization_charts.py	Membuat grafik hasil eksperimen dengan plotly, yaitu algorithm_view untuk satu *run* dan experiment_view untuk kumpulan *run*.

Alur eksekusi program adalah sebagai berikut:

1. main.py memuat masalah dengan load_problem,
2. Untuk setiap algoritma dan setiap *run*, program membuat generator acak random.Random(seed) dengan *seed* bernilai seed awal + nomor run - 1, sehingga

setiap eksperimen dapat diulang.

3. Pada algoritma *Hill-climbing* dan *Simulated Annealing*, *state* awal dibangkitkan lebih dahulu dengan generate_initial_state, sedangkan pada algoritma *Genetic Algorithm*, program membangkitkan populasi awalnya sendiri.

4. Hasil setiap *run* (nilai awal dan akhir, jumlah iterasi, durasi, alasan berhenti, dan riwayat nilai per iterasi) disimpan dalam satu berkas JSON.

3.2 Representasi State, Fungsi Objektif, dan Constraints

3.2.1 Representasi State

Peti kemas truk dimodelkan sebagai ruang tiga dimensi diskret berukuran $W \times L \times H$ satuan, dengan sumbu x sepanjang lebar (w), sumbu y sepanjang panjang (l), dan sumbu z sepanjang tinggi (h). Setiap paket berbentuk balok dengan dimensi bilangan bulat. Posisi sebuah paket adalah koordinat (x, y, z) dari sudut balok dengan nilai koordinat terkecil, sehingga paket yang diletakkan pada posisi tersebut dengan dimensi efektif (d_x, d_y, d_z) menempati rentang $[x, x+d_x) \times [y, y+d_y) \times [z, z+d_z)$.

Satu solusi atau *state* direpresentasikan oleh objek State yang memuat sebuah *dictionary* placements dengan bentuk

$$\text{placements[id_paket]} = \text{Placement(position, orientation)}$$

untuk *seluruh* paket pada masalah, baik yang dimuat maupun tidak. Atribut position berupa list [x, y, z] bila paket dimuat ke dalam truk, dan bernilai None bila paket tidak dimuat. Atribut orientation berupa permutasi dari sumbu ("x", "y", "z") yang menyatakan sumbu truk yang dipetakan berturut-turut oleh dimensi w, l, dan h milik paket.

Terdapat enam permutasi yang mungkin berdasarkan konstanta ORIENTATIONS pada package.py, misalnya orientasi ("x", "z", "y") berarti panjang paket sejajar sumbu z dan tingginya sejajar sumbu y. Dimensi efektif paket pada sumbu truk diperoleh dari Package.effective_dims. Dengan representasi ini, memuat atau mengeluarkan paket cukup dilakukan dengan mengubah position antara koordinat dan None.

Selain itu, State menyediakan metode copy untuk menyalin *state*, key untuk mengubah *state* menjadi *tuple* terurut yang dapat di-*hash* (untuk menghindari tetangga duplikat), dan to_dict untuk serialisasi ke JSON.

3.2.2 Fungsi Objektif

Tujuan masalah adalah memaksimalkan total *value* paket yang berhasil dimuat ke dalam truk. Misalkan v_i adalah *value* paket i dan pos_i adalah posisinya pada *state* S, maka *objective function* yang dimaksimalkan adalah

$$f(S) = \sum_{i\,:\,\text{pos}_i \neq None} v_i. \tag{3.1}$$

Fungsi ini diimplementasikan oleh evaluate pada objective.py (Kode 3.1). Karena seluruh *state* yang dikunjungi algoritma dijaga selalu memenuhi *constraint* (lihat Section 3.2.3 dan Section 3.3), *objective function* tidak memerlukan suku penalti pelanggaran. Pada *Genetic Algorithm*, nilai $f(S)$ digunakan langsung sebagai nilai *fitness* karena $f(S) \geq 0$ dan semakin besar semakin baik.

```
def evaluate(problem, state):
    values = {p.id: p.value for p in problem.packages}
    return sum(
        values[pid] for pid, p in state.placements.items() if p.position is
 not None
    )
```

Kode 3.1: Fungsi objektif pada objective.py

3.2.3 Constraints

Sebuah *state* dinyatakan valid apabila memenuhi lima *constraint* pada Table 3.2. Seluruh pengecekan dilakukan oleh validate_state pada constraints.py yang mengembalikan pasangan (valid, violations), dengan violations berisi daftar pesan pelanggaran. Setiap paket yang dimuat lebih dahulu diubah menjadi kotak (x, y, z, d_x, d_y, d_z) oleh collect_boxes; paket yang menembus dinding dicatat sebagai pelanggaran dan tidak diikutsertakan pada pengecekan berikutnya.

Tabel 3.2: *Constraint* dan fungsi pengecekannya

Fungsi	Deskripsi
collect_boxes(problem, state, pkgs, violations)	**Tidak menembus dinding.** Paket harus berada sepenuhnya di dalam truk: $x, y, z \geq 0$ serta $x + d_x \leq W$, $y + d_y \leq L$, dan $z + d_z \leq H$.
check_overlap(inside, violations)	**Tidak saling tumpang tindih.** Dua paket *overlap* bila rentang keduanya beririsan pada sumbu x, y, dan z sekaligus (box_overlap). Rentang dianggap setengah terbuka sehingga paket yang hanya bersentuhan pada sisinya tidak dihitung *overlap*.
check_capacity(problem, inside, pkgs, violations)	**Kapasitas berat.** Total berat seluruh paket yang dimuat tidak boleh melebihi max_capacity truk.
check_floating(inside, pkgs, violations)	**Tidak melayang.** Paket dengan $z = 0$ berada di lantai. Paket dengan $z > 0$ harus ditopang paling sedikit satu paket *non-fragile* yang sisi atasnya tepat berada pada ketinggian z dan proyeksi alasnya beririsan dengan proyeksi alas paket tersebut (footprint_overlap).
check_fragile(inside, pkgs, violations)	**Paket *fragile* tidak boleh tertindih.** Untuk setiap paket *fragile*, tidak boleh ada paket lain yang proyeksi alasnya beririsan dan posisi dasarnya berada di atas atau tepat pada sisi atas paket *fragile* tersebut. Bersama check_floating, hal ini membuat paket *fragile* tidak dapat menjadi penyangga.
validate_state(problem, state)	Menjalankan semua pengecekan di atas dan menyatakan *state* valid hanya jika violations kosong.

```
1 def overlap_1d(a0, a1, b0, b1):
2     return max(a0, b0) < min(a1, b1)
3
```

```
4
5  def footprint_overlap(box_a, box_b):
6      xa, ya, _, dxa, dya, _ = box_a
7      xb, yb, _, dxb, dyb, _ = box_b
8      return overlap_1d(xa, xa + dxa, xb, xb + dxb) and overlap_1d(ya, ya +
   dya, yb, yb + dyb)
9
10
11 def box_overlap(box_a, box_b):
12     _, _, za, _, _, dza = box_a
13     _, _, zb, _, _, dzb = box_b
14     return footprint_overlap(box_a, box_b) and overlap_1d(za, za + dza, zb,
    zb + dzb)
15
16
17 def collect_boxes(problem, state, pkgs, violations):
18     truck = problem.truck
19     inside = {}
20
21     for pid, p in state.placements.items():
22         if p.position is None:
23             continue
24         x, y, z = p.position
25         dx, dy, dz = pkgs[pid].effective_dims(p.orientation)
26         if not (x >= 0 and y >= 0 and z >= 0 and x + dx <= truck.w and y +
   dy <= truck.l and z + dz <= truck.h):
27             violations.append(f"{pid}: menembus dinding")
28             continue
29         inside[pid] = (x, y, z, dx, dy, dz)
30
31     return inside
32
33
34 def check_overlap(inside, violations):
35     ids = list(inside)
36     for a in range(len(ids)):
37         for b in range(a + 1, len(ids)):
38             i, j = ids[a], ids[b]
39             if box_overlap(inside[i], inside[j]):
40                 violations.append(f"{i}/{j}: overlap")
```

```
41
42
43  def check_capacity(problem, inside, pkgs, violations):
44      totalW = sum(pkgs[pid].weight for pid in inside)
45      if totalW > problem.truck.max_capacity:
46          violations.append(f"overload {totalW} > {problem.truck.max_capacity
    }")
47
48
49  def check_floating(inside, pkgs, violations):
50      for pid, box in inside.items():
51          z = box[2]
52          if z == 0:
53              continue
54          supported = any(
55              not pkgs[q].is_fragile
56              and other[2] + other[5] == z
57              and footprint_overlap(box, other)
58              for q, other in inside.items()
59              if q != pid
60          )
61          if not supported:
62              violations.append(f"{pid}: melayang")
63
64
65  def check_fragile(inside, pkgs, violations):
66      for pid, box in inside.items():
67          if not pkgs[pid].is_fragile:
68              continue
69          top = box[2] + box[5]
70          for q, other in inside.items():
71              if q == pid:
72                  continue
73              if footprint_overlap(box, other) and other[2] >= top:
74                  violations.append(f"{pid}: fragile tertindih {q}")
75                  break
76
77
78  def validate_state(problem, state):
79      pkgs = problem.package_map
```

```
    violations = []

    inside = collect_boxes(problem, state, pkgs, violations)
    check_overlap(inside, violations)
    check_capacity(problem, inside, pkgs, violations)
    check_floating(inside, pkgs, violations)
    check_fragile(inside, pkgs, violations)

    return (len(violations) == 0, violations)
```

Kode 3.2: Validasi *constraint* pada constraints.py

3.3 Representasi Neighbor dan State Awal

3.3.1 Pembangkitan State Awal

Pada *Hill-climbing* dan *Simulated Annealing*, *state* awal dibangkitkan oleh generate_initial_state pada initialization.py (Kode 3.3). Fungsi ini dimulai dari *state* kosong, yaitu semua paket berposisi None. Urutan paket diacak, dan untuk setiap paket urutan keenam orientasinya juga diacak. Pada setiap orientasi, seluruh posisi bulat yang membuat paket muat di dalam dimensi truk (candidate_positions) diacak, lalu dicoba satu per satu; posisi pertama yang membuat seluruh *state* tetap valid menurut validate_state dipakai. Jika tidak ada kombinasi orientasi dan posisi yang valid, paket dibiarkan di luar truk. Dengan demikian, *state* awal selalu valid, tetapi tidak dioptimalkan sama sekali dan dapat berbeda pada setiap *seed*.

```
from src.constraints import validate_state
from src.neighbors import candidate_positions
from src.package import ORIENTATIONS
from src.placement import Placement
from src.state import State


def generate_initial_state(problem, rng):
    pkgs = problem.package_map
    state = State(
        {pid: Placement(None, ["x", "y", "z"]) for pid in pkgs}
    )
    order = list(pkgs)
```

```
    rng.shuffle(order)
    for pid in order:
        oris = list(ORIENTATIONS)
        rng.shuffle(oris)
        placed = False
        for ori in oris:
            positions = list(candidate_positions(problem, pkgs[pid], ori))
            rng.shuffle(positions)
            for pos in positions:
                cand = state.copy()
                cand.placements[pid] = Placement(pos, list(ori))
                valid, _ = validate_state(problem, cand)
                if valid:
                    state = cand
                    placed = True
                    break
            if placed:
                break
    return state
```

Kode 3.3: Pembangkitan *state* awal pada initialization.py

Pada *Genetic Algorithm*, *state* awal berupa populasi yang tiap individunya adalah *state* lengkap. Individu dibangkitkan oleh metode GeneticAlgorithm._generate_initial_state yang menyisipkan paket secara acak satu per satu dengan pengecekan *constraint* inkremental. Individu terbaik pada populasi awal dicatat sebagai initial_state pada hasil *run*.

3.3.2 Operasi Tetangga

Tetangga dari sebuah *state* adalah *state* baru yang diperoleh dengan tepat satu langkah perubahan terhadap *state* tersebut dan memenuhi seluruh *constraint*. Terdapat tiga operator pada neighbors.py:

1. **Move**: memindahkan satu paket ke koordinat (x, y, z) lain dengan orientasi tetap. Jika paket yang dipilih sebelumnya berada di luar truk, operator ini sekaligus memuatnya ke dalam truk.

2. **Rotate**: merotasi satu paket sebesar 90° terhadap sumbu x, y, atau z dengan posisi tetap. Rotasi terhadap suatu sumbu menukar dua sumbu lainnya pada

orientation (Placement.rotate).

3. **Swap**: menukar posisi dua paket, baik keduanya di dalam truk maupun salah satunya di luar truk. Orientasi masing-masing paket tidak berubah. Pasangan yang keduanya berada di luar truk dilewati karena tidak mengubah *state*.

Paket hanya dapat dikeluarkan dari truk melalui operator *swap* dengan paket yang berada di luar truk. Semua tetangga yang sama dengan *state* saat ini atau dengan tetangga yang sudah dihasilkan sebelumnya dibuang menggunakan himpunan seen yang berisi State.key(), dan tetangga yang melanggar *constraint* dibuang oleh is_new_valid.

Terdapat dua cara pembangkitan tetangga, yaitu:

- Fungsi iter_valid_neighbors menghasilkan *seluruh* tetangga valid secara berurutan (operator *move*, kemudian *rotate*, kemudian *swap*) dan digunakan oleh varian *Hill-climbing* yang memerlukan seluruh tetangga, yaitu *steepest ascent*, *sideways move*, dan *random restart*.

- Fungsi sample_random_neighbor menghasilkan *satu* tetangga valid secara acak dan digunakan oleh *stochastic Hill-climbing* serta *Simulated Annealing*: urutan ketiga operator diacak, lalu untuk setiap operator dicoba hingga max_attempts (bawaan 50) kandidat acak, dan kandidat valid pertama dikembalikan. Fungsi ini mengembalikan None apabila tidak ditemukan tetangga valid.

Pada *Genetic Algorithm*, konsep tetangga digantikan oleh operator *crossover* dan *mutation*. Operator *mutation* memakai tiga jenis perubahan yang sama dengan operator tetangga di atas.

```
from src.constraints import validate_state
from src.package import ORIENTATIONS
from src.placement import Placement


def candidate_positions(problem, package, orientation):
    truck = problem.truck
    dx, dy, dz = package.effective_dims(orientation)
    W, L, H = truck.w, truck.l, truck.h
    if dx > W or dy > L or dz > H:
        return
    for x in range(W - dx + 1):
```

```
        for y in range(L - dy + 1):
            for z in range(H - dz + 1):
                yield [x, y, z]


def is_new_valid(problem, cand, seen):
    key = cand.key()
    if key in seen:
        return False
    seen.add(key)
```

Kode 3.4: Fungsi bantu pembangkit tetangga pada neighbors.py

```


def iter_valid_neighbors(problem, state):
    pkgs = problem.package_map
    ids = list(state.placements)
    seen = {state.key()}

    # Move: pindahkan satu paket ke koordinat lain, orientasi tetap
    for pid in ids:
        cur = state.placements[pid]
        for pos in candidate_positions(problem, pkgs[pid], cur.orientation)
:
            cand = state.copy()
            cand.placements[pid].position = pos
            if is_new_valid(problem, cand, seen):
                yield cand

    # Rotate: rotasi satu paket 90° terhadap sumbu x, y, atau z, posisi
tetap
    for pid in ids:
        for axis in ("x", "y", "z"):
            cand = state.copy()
            cand.placements[pid].rotate(axis)
            if is_new_valid(problem, cand, seen):
                yield cand

    # Swap: tukar posisi dua paket, di dalam maupun di luar truk
```

```
    for a in range(len(ids)):
        for b in range(a + 1, len(ids)):
            i, j = ids[a], ids[b]
            pi, pj = state.placements[i].position, state.placements[j].
position
            if pi is None and pj is None:
                continue
            cand = state.copy()
            cand.placements[i].position = list(pj) if pj is not None else
None
            cand.placements[j].position = list(pi) if pi is not None else
None
```

Kode 3.5: Pembangkitan seluruh tetangga valid (iter_valid_neighbors)

```
                yield cand

# Ambil satu tetangga acak yang valid dari state
def sample_random_neighbor(problem, state, rng, max_attempts=50):
    pkgs = problem.package_map
    ids = list(state.placements)
    seen = {state.key()}
    operators = [0, 1, 2]
    rng.shuffle(operators)

    for op in operators:
        for _ in range(max_attempts):
            if op == 0:
                # Move
                pid = rng.choice(ids)
                cur = state.placements[pid]
                positions = list(candidate_positions(problem, pkgs[pid],
cur.orientation))
                if not positions:
                    continue
                pos = rng.choice(positions)
                cand = state.copy()
                cand.placements[pid].position = pos
                if is_new_valid(problem, cand, seen):
                    return cand
```

```
25
26              elif op == 1:
27                  # Rotate
28                  pid = rng.choice(ids)
29                  axis = rng.choice(("x", "y", "z"))
30                  cand = state.copy()
31                  cand.placements[pid].rotate(axis)
32                  if is_new_valid(problem, cand, seen):
33                      return cand
34
35              else:
36                  # Swap
37                  if len(ids) < 2:
38                      continue
39                  i, j = rng.sample(ids, 2)
40                  pi = state.placements[i].position
41                  pj = state.placements[j].position
42                  if pi is None and pj is None:
43                      continue
44                  cand = state.copy()
45                  cand.placements[i].position = list(pj) if pj is not None
    else None
46                  cand.placements[j].position = list(pi) if pi is not None
    else None
47                  if is_new_valid(problem, cand, seen):
48                      return cand
```

Kode 3.6: Pengambilan satu tetangga acak valid (sample_random_neighbor)

3.4 Implementasi Kelas-kelas Pendukung

3.4.1 truck.py

Tabel 3.3: Kelas/fungsi pada truck.py

Kelas/Fungsi	Deskripsi
class Truck	Merepresentasikan peti kemas truk.

Kelas/Fungsi	Deskripsi
__init__(self, w, l, h, max_capacity)	Menyimpan dimensi truk (lebar w, panjang l, tinggi h) dan kapasitas berat maksimum max_capacity.

```python
class Truck:
    def __init__(self, w, l, h, max_capacity):
        self.w = w
        self.l = l
        self.h = h
        self.max_capacity = max_capacity

    def print(self):
        print(f"Truck(w={self.w}, l={self.l}, h={self.h}, max_capacity={self.max_capacity})")
```

Kode 3.7: Kelas Truck

3.4.2 package.py

Tabel 3.4: Kelas/fungsi pada package.py

Kelas/Fungsi	Deskripsi
ORIENTATIONS	Daftar enam permutasi sumbu ("x", "y", "z") yang merepresentasikan seluruh orientasi balok.
class Package	Merepresentasikan satu paket beserta atributnya.
__init__(self, id, w, l, h, value, weight, is_fragile, eta)	Menyimpan identitas, dimensi, *value*, berat, status *fragile*, dan ETA paket. Atribut eta dibaca dari masukan tetapi tidak digunakan pada *objective function* maupun *constraint* implementasi ini.

Kelas/Fungsi	Deskripsi
effective_dims(self, orientation)	Mengembalikan dimensi paket (dx, dy, dz) pada sumbu x, y, z truk untuk sebuah orientasi, dengan memetakan dimensi w, l, h paket ke sumbu yang ditunjuk orientasi.

```python
ORIENTATIONS = [
    ("x", "y", "z"),
    ("x", "z", "y"),
    ("y", "x", "z"),
    ("z", "x", "y"),
    ("y", "z", "x"),
    ("z", "y", "x"),
]


class Package:
    def __init__(self, id, w, l, h, value, weight, is_fragile, eta):
        self.id = id
        self.w = w
        self.l = l
        self.h = h
        self.value = value
        self.weight = weight
        self.is_fragile = is_fragile
        self.eta = eta

    def print(self):
        print(f"Package(id={self.id}, dims=({self.w},{self.l},{self.h}), value={self.value}, weight={self.weight}, fragile={self.is_fragile}, eta={self.eta})")

    def effective_dims(self, orientation):
        w_axis, l_axis, h_axis = orientation
        size = {w_axis: self.w, l_axis: self.l, h_axis: self.h}
        return (size["x"], size["y"], size["z"])
```

Kode 3.8: Kelas Package

3.4.3 problem.py

Tabel 3.5: Kelas/fungsi pada problem.py

Kelas/Fungsi	Deskripsi
class Problem	Menyatukan truk dan seluruh paket menjadi satu instans masalah.
__init__(self, truck, packages)	Menyimpan objek Truck, daftar Package, dan package_map yang memetakan id paket ke objek paket untuk akses cepat.

```
class Problem:
    def __init__(self, truck, packages):
        self.truck = truck
        self.packages = packages
        self.package_map = {p.id: p for p in packages}

    def print(self):
        self.truck.print()
        for p in self.packages:
            p.print()
```

Kode 3.9: Kelas Problem

3.4.4 placement.py

Tabel 3.6: Kelas/fungsi pada placement.py

Kelas/Fungsi	Deskripsi
class Placement	Merepresentasikan penempatan satu paket.
__init__(self, position, orientation)	Menyimpan posisi [x, y, z] (atau None jika paket tidak dimuat) dan orientasi paket.
rotate(self, axis)	Merotasi paket 90° terhadap sumbu axis dengan menukar dua sumbu lainnya pada orientation.

```python
class Placement:
    def __init__(self, position, orientation):
        self.position = position
        self.orientation = orientation

    def print(self):
        print(f"Placement(position={self.position}, orientation={self.orientation})")

    def rotate(self, axis):
        # rotasi 90° terhadap suatu sumbu = dua sumbu lainnya bertukar
        a, b = {"x": ("y", "z"), "y": ("x", "z"), "z": ("x", "y")}[axis]
        self.orientation = [
            b if s == a else a if s == b else s
            for s in self.orientation
        ]
```

Kode 3.10: Kelas Placement

3.4.5 state.py

Tabel 3.7: Kelas/fungsi pada state.py

Kelas/Fungsi	**Deskripsi**
class State	Merepresentasikan satu solusi, yaitu penempatan seluruh paket.
__init__(self, placements)	Menyimpan *dictionary* id paket ke Placement.
copy(self)	Membuat salinan mendalam *state*, termasuk list posisi dan orientasi, agar perubahan pada tetangga tidak memengaruhi *state* asal.
key(self)	Mengubah *state* menjadi *tuple* terurut berdasarkan id paket yang dapat di-*hash*, sehingga dapat disimpan pada himpunan untuk mendeteksi *state* yang sama.
to_dict(self)	Mengubah *state* menjadi *dictionary* yang dapat diserialisasi ke JSON.

```python
from src.placement import Placement


class State:
    def __init__(self, placements):
        self.placements = placements

    def print(self):
        print("State(")
        for pid, p in self.placements.items():
            print(f"  {pid}: position={p.position}, orientation={p.orientation}")
        print(")")

    def copy(self):
        return State({
            pid: Placement(
                list(p.position) if p.position is not None else None,
                list(p.orientation),
            )
            for pid, p in self.placements.items()
        })

    # menyimpan dict sebuah state ke dalam bentuk tuple
    def key(self):
        return tuple(
            sorted(
                (
                    pid,
                    tuple(p.position) if p.position is not None else None,
                    tuple(p.orientation),
                )
                for pid, p in self.placements.items()
            )
        )

    def to_dict(self):
        return {
            "placements": {
```

```
                pid: {"position": p.position, "orientation": p.orientation}
                for pid, p in self.placements.items()
            }
        }

    def occupancy(self, problem):
        pkgs = problem.package_map
        occ = {}
        for pid, p in self.placements.items():
            if p.position is None:
                continue
            dx, dy, dz = pkgs[pid].effective_dims(p.orientation)
            x, y, z = p.position
            for i in range(x, x + dx):
                for j in range(y, y + dy):
                    for k in range(z, z + dz):
                        occ[(i, j, k)] = pid
        return occ
```

Kode 3.11: Kelas State

3.4.6 reader.py

Tabel 3.8: Fungsi pada reader.py

Kelas/Fungsi	Deskripsi
load_problem(path)	Membuka berkas JSON pada path, membentuk objek Truck dari data truck (dimensi dan max_capacity), membentuk satu objek Package untuk setiap entri packages, lalu mengembalikan objek Problem.

```
import json

from src.package import Package
from src.problem import Problem
from src.truck import Truck

```

```

def load_problem(path):
    with open(path) as f:
        data = json.load(f)
    td = data["truck"]["dimensions"]
    truck = Truck(td["w"], td["l"], td["h"], data["truck"]["max_capacity"])
    packages = [
        Package(p["id"], p["dimensions"]["w"], p["dimensions"]["l"], p["dimensions"]["h"],
                p["value"], p["weight"], p["is_fragile"], p["eta"])
        for p in data["packages"]
    ]
    return Problem(truck, packages)
```

Kode 3.12: Fungsi load_problem

3.4.7 common.py

Tabel 3.9: Fungsi pada common.py

Kelas/Fungsi	Deskripsi
ser(s)	Mengubah objek State menjadi *dictionary* melalui to_dict; nilai yang sudah berupa *dictionary* dikembalikan apa adanya.
build_result(...)	Menyusun *dictionary* hasil *run* dengan format seragam untuk semua algoritma: nama algoritma, parameter, *state* dan nilai awal, *state* dan nilai akhir, jumlah iterasi, jumlah perpindahan yang diterima, jumlah evaluasi *objective function*, durasi, alasan berhenti, riwayat per iterasi, dan informasi tambahan khusus algoritma (extra).

```


def _ser(s):
    return s.to_dict() if hasattr(s, "to_dict") else s
```

```
5
6
7  def build_result(
8      algorithm,
9      parameters,
10     initial_state,
11     initial_value,
12     final_state,
13     final_value,
14     iterations,
15     accepted_moves,
16     objective_evaluations,
17     duration_seconds,
18     stop_reason,
19     history,
20     extra,
21 ):
22     return {
23         "algorithm": algorithm,
24         "parameters": parameters,
25         "initial_state": _ser(initial_state),
26         "final_state": _ser(final_state),
27         "initial_value": initial_value,
28         "final_value": final_value,
29         "iterations": iterations,
30         "accepted_moves": accepted_moves,
31         "objective_evaluations": objective_evaluations,
32         "duration_seconds": duration_seconds,
33         "stop_reason": stop_reason,
34         "history": history,
35         "extra": extra,
36     }
```

Kode 3.13: Fungsi pada common.py

3.5 Implementasi Algoritma Hill-climbing

3.5.1 hill_climb.py

Seluruh varian *Hill-climbing* diimplementasikan sebagai metode pada kelas HillClimb dan memaksimalkan *objective function*. Ke-empat varian menerima *state* awal sebagai argumen dan mengembalikan hasil dalam format build_result.

Tabel 3.10: Kelas/fungsi pada hill_climb.py

Kelas/Fungsi	**Deskripsi**
class HillClimb	Pembungkus seluruh varian *Hill-climbing* untuk sebuah Problem.
__init__(self, problem, rng)	Menyimpan instans masalah dan generator acak rng yang digunakan untuk memilih di antara tetangga yang nilainya sama dan untuk pembangkitan tetangga acak.
steepest_ascent(self, initial_state)	Pada setiap iterasi, membangkitkan seluruh tetangga valid, lalu berpindah ke tetangga dengan nilai tertinggi bila nilainya lebih besar dari *state* saat ini. Jika beberapa tetangga bernilai tertinggi sama, salah satunya dipilih secara acak. Berhenti jika tidak ada tetangga yang lebih baik.
sideways(self, initial_state, max_sideways=10)	Serupa dengan *steepest ascent*, tetapi jika tidak ada tetangga yang lebih baik dan terdapat tetangga bernilai sama, pencarian tetap berpindah ke salah satunya (*sideways move*) selama jumlah *sideways move* belum mencapai max_sideways.

Dilanjutkan ke halaman berikutnya

Tabel 3.10 – lanjutan

Kelas/Fungsi	**Deskripsi**
random_restart(self, initial_state, max_restart=3)	Menjalankan *steepest ascent* dari *state* awal yang diberikan, kemudian dari max_restart *state* awal acak baru, lalu mengambil hasil dengan nilai akhir tertinggi. Riwayat seluruh pencarian digabung pada satu sumbu iterasi global.
stochastic(self, initial_state, max_iterations=10_000)	Pada setiap iterasi mengambil satu tetangga acak valid dan berpindah hanya jika nilainya lebih baik. Berhenti setelah max_iterations iterasi atau jika tidak ditemukan tetangga valid.

Kelas HillClimb dan varian *steepest ascent* ditunjukkan pada Kode 3.14. Pada setiap iterasi, iter_valid_neighbors dievaluasi satu per satu; variabel best_value diinisialisasi dengan nilai *state* saat ini sehingga best_list hanya memuat tetangga yang nilainya lebih besar atau sama dengan *state* saat ini. Pencarian berhenti dengan alasan no_improving_neighbor bila tidak ada tetangga yang lebih baik, atau no_valid_neighbors bila tidak ada satu pun tetangga yang nilainya sama maupun lebih baik.

```python
import time

from src.algorithms.common import build_result
from src.neighbors import iter_valid_neighbors, sample_random_neighbor
from src.objective import evaluate


class HillClimb:
    def __init__(self, problem, rng):
        self.problem = problem
        self.rng = rng

    def print(self):
        print(f"HillClimb(packages={len(self.problem.packages)})")

```

```
    def steepest_ascent(self, initial_state):
        # referensi: materi kuliah AI BeyondClassicalSearch bagian Stochastic Hill-climbing
        # setiap state menhidupi semua successor dan cari succesor yang menghasilkan state value terbaik. Jika tidak ada successor yang lebih baik, berhenti.
        t0 = time.perf_counter()
        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        history = [{"iteration": 0, "current_value": current_value, "best_value": current_value}]
        accepted = 0
        iteration = 0

        while True:
            # cari semua tetangga yang valid, pilih yang terbaik
            best_value, best_list = current_value, []
            for cand in iter_valid_neighbors(self.problem, current):
                v = evaluate(self.problem, cand)
                evals += 1
                if v > best_value:
                    best_value, best_list = v, [cand]
                elif v == best_value:
                    best_list.append(cand)
            # jika ada tetangga yang lebih baik, pindah ke salah satunya. Jika tidak ada, berhenti.
            if best_value > current_value:
                current = self.rng.choice(best_list) if len(best_list) > 1 else best_list[0]
                current_value = best_value
                accepted += 1
                iteration += 1
                history.append(
                    {"iteration": iteration, "current_value": current_value, "best_value": current_value}
                )
            else:
                iteration += 1
```

```
                history.append(
                    {"iteration": iteration, "current_value": current_value
, "best_value": current_value}
                )
                stop = "no_valid_neighbors" if not best_list else "
no_improving_neighbor"
                break

        return build_result(
            "steepest_ascent", {}, start, history[0]["current_value"],
```

Kode 3.14: Kelas HillClimb dan *steepest ascent*

Varian *sideways move* pada Kode 3.15 menambahkan penghitung sideways. Perpindahan ke tetangga bernilai sama hanya dilakukan bila tidak ada tetangga yang lebih baik dan penghitung belum mencapai max_sideways. Penghitung ini dihitung kumulatif sepanjang pencarian dan tidak di-*reset* ketika ditemukan perbaikan. Pencarian berhenti dengan alasan sideways_limit_reached ketika batas tercapai.

```
            time.perf_counter() - t0, stop, history, {},
        )

    def sideways(self, initial_state, max_sideways=10):
        # referensi: materi kuliah AI BeyondClassicalSearch bagian
Stochastic Hill-climbing
        # sama seperti steepest ascent, tapi jika ada tetangga yang sama
baiknya dengan
        t0 = time.perf_counter()
        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        history = [{"iteration": 0, "current_value": current_value, "
best_value": current_value}]
        accepted = sideways = iteration = 0

        while True:
            best_value, best_list = current_value, []
            for cand in iter_valid_neighbors(self.problem, current):
                v = evaluate(self.problem, cand)
```

```
                evals += 1
                if v > best_value:
                    best_value, best_list = v, [cand]
                elif v == best_value:
                    best_list.append(cand)
            # jika ada tetangga yang lebih baik, pindah ke salah satunya. Jika tidak ada, tapi ada tetangga yang sama baiknya dan belum melewati batas sideways, pindah ke salah satunya. Jika tidak ada tetangga yang lebih baik atau sama baiknya, berhenti.
            if best_value > current_value:
                move, sideways_move = True, False
            # jika ada tetangga yang sama baiknya, pindah ke salah satunya, iterasi sideways bertambah 1.
            elif best_list and sideways < max_sideways:
                move, sideways_move = True, True
            else:
                move = False
            iteration += 1
            if not move:
                history.append(
                    {"iteration": iteration, "current_value": current_value, "best_value": current_value}
                )
                if not best_list:
                    stop = "no_valid_neighbors"
                elif sideways >= max_sideways:
                    stop = "sideways_limit_reached"
                else:
                    stop = "no_improving_neighbor"
                break
            current = self.rng.choice(best_list) if len(best_list) > 1 else best_list[0]
            current_value = best_value
            accepted += 1
            sideways += sideways_move
            history.append(
                {"iteration": iteration, "current_value": current_value, "best_value": current_value,
                 "sideways": sideways_move}
            )
```

```
            if sideways_move and sideways >= max_sideways:
                stop = "sideways_limit_reached"
                break

        return build_result(
            "sideways", {"maximum_sideways_move": max_sideways},
            start, history[0]["current_value"], current, current_value,
            {"maximum_sideways_move": max_sideways, "sideways_moves":
sideways},
        )

    def random_restart(self, initial_state, max_restart=3):
        from src.initialization import generate_initial_state
        # referensi: materi kuliah AI BeyondClassicalSearch bagian
Stochastic Hill-climbing
        # jalankan steepest ascent beberapa kali dari initial state yang
berbeda, ambil hasil terbaik

        t0 = time.perf_counter()
        starts = [initial_state.copy()] + [
            generate_initial_state(self.problem, self.rng) for _ in range(
max_restart)
        ]
        runs = [self.__class__(self.problem, self.rng).steepest_ascent(s)
for s in starts]
        best_idx = max(range(len(runs)), key=lambda i: runs[i]["final_value
"])
        best = runs[best_idx]
        searches = [
            {
                "search_id": i,
```

Kode 3.15: Varian *sideways move*

Varian *random restart* pada Kode 3.16 menjalankan *steepest ascent* sebanyak max_restart +1 kali: satu kali dari *state* awal yang diberikan dan sisanya dari *state* awal baru yang dibangkitkan generate_initial_state. Jumlah iterasi, perpindahan yang diterima, dan evaluasi dijumlahkan dari seluruh pencarian, sedangkan *state* akhir diambil dari pencarian dengan nilai akhir tertinggi.

```
                "initial_state": starts[i].to_dict(),
                "final_state": r["final_state"],
                "initial_value": r["initial_value"],
                "final_value": r["final_value"],
                "iterations": r["iterations"],
                "accepted_moves": r["accepted_moves"],
                "stop_reason": r["stop_reason"],
                "history": r["history"],
            }
            for i, r in enumerate(runs)
        ]

        # gabungin riwayat semua pencarian dalam satu sumbu iterasi global
        history, offset, global_best = [], 0, None
        for i, r in enumerate(runs):
            for h in r["history"]:
                if i > 0 and h["iteration"] == 0:
                    offset += 1
                global_best = h["current_value"] if global_best is None
else max(global_best, h["current_value"])
                history.append({
                    "iteration": offset + h["iteration"],
                    "search_id": i,
                    "local_iteration": h["iteration"],
                    "current_value": h["current_value"],
                    "best_value": global_best,
                })
            offset += r["history"][-1]["iteration"]
        total_iterations = sum(r["iterations"] for r in runs)

        return build_result(
            "random_restart", {"maximum_restart": max_restart},
            starts[0].copy(), runs[0]["initial_value"],
            best["final_state"], best["final_value"],
            total_iterations, sum(r["accepted_moves"] for r in runs),
            sum(r["objective_evaluations"] for r in runs),
            time.perf_counter() - t0, best["stop_reason"], history,
            {
                "maximum_restart": max_restart,
                "num_searches": len(runs),
```

```
                "restart_count": len(runs) - 1,
                "best_search": best_idx,
                "total_iterations": total_iterations,
                "iterations_per_search": [r["iterations"] for r in runs],
```

Kode 3.16: Varian *random restart*

Varian *stochastic* pada Kode 3.17 tidak mengevaluasi seluruh tetangga, melainkan hanya satu tetangga acak per iterasi yang diterima bila nilainya lebih tinggi dari *state* saat ini (delta > 0). *State* terbaik dicatat terpisah pada variabel best.

```
            },
        )

    def stochastic(self, initial_state, max_iterations=10_000):
        # ambil satu  neighbour acak, pindah hanya jika lebih baik dari state saat ini
        # referensi: materi kuliah AI BeyondClassicalSearch bagian Stochastic Hill-climbing
        t0 = time.perf_counter()
        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        best = current.copy()
        best_value = current_value
        accepted = iteration = 0
        history = [
            {"iteration": 0, "current_value": current_value, "best_value": best_value, "accepted": False}
        ]
        stop = "max_iterations"

        while iteration < max_iterations:
            neighbour = sample_random_neighbor(self.problem, current, self.rng)
            if neighbour is None:
                stop = "no_valid_neighbors"
                break

```

```
            neighbour_value = evaluate(self.problem, neighbour)
            evals += 1
            delta = neighbour_value - current_value

            is_accepted = delta > 0
            if is_accepted:
                current = neighbour
                current_value = neighbour_value
                accepted += 1
                if current_value > best_value:
                    best = current.copy()
                    best_value = current_value

            iteration += 1
            history.append(
                {
                    "iteration": iteration,
                    "current_value": current_value,
                    "best_value": best_value,
                    "accepted": is_accepted,
                }
            )

        return build_result(
            "stochastic", {"max_iterations": max_iterations},
            start, history[0]["current_value"], best, best_value,
```

Kode 3.17: Varian *stochastic*

3.6 Implementasi Algoritma Simulated Annealing

3.6.1 simulated_annealing.py

Tabel 3.11: Kelas/fungsi pada simulated_annealing.py

Kelas/Fungsi	**Deskripsi**
class SimulatedAnnealing	Implementasi *Simulated Annealing* untuk memaksimalkan *objective function*.
__init__(self, problem, rng)	Menyimpan instans masalah dan generator acak.
run(self, initial_state, initial_temperature=100.0, minimum_temperature=0.1, cooling_rate=0.995, max_iterations=10_000, stuck_patience=100)	Menjalankan pencarian dari *state* awal. Pada setiap iterasi diambil satu tetangga acak valid; tetangga yang lebih baik selalu diterima, sedangkan tetangga yang sama atau lebih buruk diterima dengan probabilitas $e^{\Delta E/T}$. Suhu T diturunkan secara geometris dengan faktor cooling_rate. Selain itu, dicatat jumlah kejadian *stuck* dan jumlah perpindahan buruk yang diterima maupun ditolak.

Pada metode run, *state* saat ini (current) dan *state* terbaik yang pernah dikunjungi (best) disimpan terpisah, karena penerimaan perpindahan yang lebih buruk dapat menurunkan nilai *state* saat ini. Hasil akhir yang dikembalikan adalah best. Kode 3.18 menunjukkan parameter, inisialisasi variabel pencacah, dan format riwayat per iterasi.

```
import math
import time

from src.algorithms.common import build_result
from src.neighbors import sample_random_neighbor
from src.objective import evaluate


class SimulatedAnnealing:

    def __init__(self, problem, rng):
        self.problem = problem
        self.rng = rng

```

```
    def print(self):
        print(f"SimulatedAnnealing(packages={len(self.problem.packages)})")

    def run(
        self,
        initial_state,
        initial_temperature=100.0,
        minimum_temperature=0.1,
        cooling_rate=0.995,
        max_iterations=10_000,
        stuck_patience=100, # default
    ):
        t0 = time.perf_counter()

        start = initial_state.copy()
        current = initial_state.copy()
        current_value = evaluate(self.problem, current)
        evals = 1
        best = current.copy()
        best_value = current_value
        temperature = initial_temperature
        iteration = 0
        accepted = 0
        bad_accepted = 0
        bad_rejected = 0
        # definisi stuck: state dianggap terjebak di local optimum jika selama stuck_patience iterasi berturut-turut tidak ada satu pun tetangga acak yang lebih baik dari state saat ini. Satu episode stuck dihitung sekali dan dianggap lolos ketika akhirnya ditemukan tetangga yang lebih baik.
        stuck_count = 0
        stuck_iterations = 0
        no_improve_streak = 0
        stuck_events = []

        history = [
            {
                "iteration": 0,
                "temperature": temperature,
                "current_value": current_value,
```

```
                "best_value": best_value,
                "delta_e": None,
                "exp_delta_e_over_t": None,
                "acceptance_probability": None,
                "stuck": False,
            }
        ]
```

Kode 3.18: Parameter dan inisialisasi *Simulated Annealing*

Perulangan utama pada Kode 3.19 berjalan selama jumlah iterasi belum mencapai max_iterations dan suhu masih lebih besar atau sama dengan minimum_temperature. Pada setiap iterasi dihitung $\Delta E = f(\text{tetangga}) - f(\text{saat ini})$. Jika $\Delta E > 0$, tetangga diterima dengan probabilitas 1. Jika tidak, tetangga diterima dengan probabilitas

$$P = e^{\Delta E / T} \tag{3.2}$$

dengan membandingkan P terhadap bilangan acak seragam pada $[0, 1)$ (aturan penerimaan Metropolis). Karena $\Delta E = 0$ menghasilkan $P = 1$, tetangga bernilai sama selalu diterima. Pembatas min(delta / temperature, 700.0) pada pemanggilan math.exp dipakai untuk mencegah *overflow* numerik. Setelah setiap iterasi, suhu diperbarui dengan $T \leftarrow \alpha T$, dengan α adalah cooling_rate.

Selain itu, perulangan melacak kejadian *stuck*. Pencarian dianggap *stuck* apabila terdapat stuck_patience iterasi berturut-turut tanpa tetangga yang lebih baik dari *state* saat ini. Setiap episode *stuck* dihitung satu kali pada stuck_count, dicatat pada stuck_events beserta iterasi deteksi dan iterasi lolosnya, dan dianggap lolos ketika akhirnya ditemukan tetangga yang lebih baik.

```

        stop_reason = "max_iterations"

        # reference to materi kuliah AI BeyondClassicalSearch, tp jika ada yg diluar itu maka akan ditulis secara explicit
        while iteration < max_iterations and temperature >= minimum_temperature:
            neighbour = sample_random_neighbor(self.problem, current, self.rng)

            if neighbour is None:
```

```
                stop_reason = "no_valid_neighbors"
                break

            neighbour_value = evaluate(self.problem, neighbour)
            evals += 1
            delta = neighbour_value - current_value
            exp_value = math.exp(delta / temperature) if delta <= 0 else None

            if delta > 0:
                # gerakan lebih baik selalu diterima (reference to materi kuliah AI BeyondClassicalSearch)
                acceptance_prob = 1.0
                current = neighbour
                current_value = neighbour_value
                accepted += 1

                if current_value > best_value:
                    best = current.copy()
                    best_value = current_value
            else:
                # gerakan sama atau lebih buruk bisa diterima dengan probabilitas (reference to materi kuliah AI BeyondClassicalSearch)
                acceptance_prob = exp_value

                # aturan penerimaan acak (Metropolis acceptance rule utk SA)
                # reference: S. Kirkpatrick, C. D. Gelatt, Jr., dan M. P. Vecchi, "Optimization by Simulated Annealing", Science, 1983. https://doi.org/10.1126/science.220.4598.671
                if self.rng.random() < acceptance_prob:
                    current = neighbour
                    current_value = neighbour_value
                    accepted += 1
                    bad_accepted += 1
                else:
                    bad_rejected += 1

            iteration += 1
            if delta > 0:
```

```
                if no_improve_streak >= stuck_patience:
                    stuck_events[-1]["escaped_at"] = iteration
                no_improve_streak = 0
            else:
                no_improve_streak += 1
                if no_improve_streak == stuck_patience:
                    stuck_count += 1
                    stuck_events.append({
                        "detected_at": iteration,
                        "value": current_value,
                        "escaped_at": None,
                    })
            is_stuck = no_improve_streak >= stuck_patience
            stuck_iterations += is_stuck

            history.append(
                {
                    "iteration": iteration,
                    "temperature": round(temperature, 6),
                    "current_value": current_value,
                    "best_value": best_value,
                    "delta_e": delta,
                    "exp_delta_e_over_t": None if exp_value is None else
round(exp_value, 6),
                    "acceptance_probability": round(acceptance_prob, 6),
                    "stuck": is_stuck,
```

Kode 3.19: Perulangan utama *Simulated Annealing*

Pencarian berhenti karena salah satu dari tiga alasan, yaitu max_iterations (batas iterasi tercapai), temperature_minimum_reached (suhu turun di bawah suhu minimum), atau no_valid_neighbors (tidak ditemukan tetangga valid). Dengan parameter bawaan ($T_0 = 100$, $T_{\min} = 0.1$, $\alpha = 0.995$), suhu turun di bawah suhu minimum setelah 1.379 iterasi, sehingga pencarian umumnya berhenti karena suhu, bukan karena batas iterasi 10.000. Kode 3.20 menunjukkan penyusunan hasil akhir.

```
            )
            temperature *= cooling_rate

        if stop_reason == "max_iterations" and temperature <
```

```
minimum_temperature:
            stop_reason = "temperature_minimum_reached"

        params = {
            "initial_temperature": initial_temperature,
            "minimum_temperature": minimum_temperature,
            "cooling_rate": cooling_rate,
            "max_iterations": max_iterations,
            "stuck_patience": stuck_patience,
        }
        extra = {
            **params,
            "stuck_definition": (
                f"{stuck_patience} proposal berturut-turut tanpa tetangga yang lebih baik"
            ),
            "stuck_count": stuck_count,
            "stuck_iterations": stuck_iterations,
            "stuck_events": stuck_events,
            "bad_accepted": bad_accepted,
            "bad_rejected": bad_rejected,
        }

        return build_result(
            "simulated_annealing",
            params,
            start,
            history[0]["current_value"],
            best,
            best_value,
            iteration,
            accepted,
            evals,
            time.perf_counter() - t0,
            stop_reason,
```

Kode 3.20: Penentuan alasan berhenti dan penyusunan hasil *Simulated Annealing*

3.7 Implementasi Algoritma Genetic Algorithm

3.7.1 genetic_algorithm.py

Genetic Algorithm yang diimplementasikan menggunakan State sebagai individu (kromosom), dengan Placement setiap paket sebagai gen. Populasi dievolusikan selama sejumlah iterasi (generasi) tertentu melalui seleksi orang tua, *crossover*, dan *mutation*. Berbeda dengan algoritma lain, *Genetic Algorithm* membangun individu baru dengan pengecekan *constraint* inkremental pada metode _can_insert dan _insert, sehingga setiap individu pada populasi selalu valid.

Tabel 3.12: Kelas/fungsi pada genetic_algorithm.py

Kelas/Fungsi	**Deskripsi**
class GeneticAlgorithm	Implementasi *Genetic Algorithm* yang menyediakan siklus seleksi, *crossover*, dan *mutation*. Konstanta kelas: CROSSOVER_PROBABILITY = 0,80, MUTATION_PROBABILITY = 0,30, REPAIR_ATTEMPTS = 20, dan MUTATION_ATTEMPTS = 10.
__init__(self, problem, rng)	Menyimpan instans masalah, generator acak, dan package_map.
random_inside_position(self, package, orientation, rng, heights)	Mengembalikan posisi acak yang membuat paket berada di dalam dimensi truk, atau None bila paket tidak muat pada orientasi tersebut. Jika heights diberikan, koordinat z dipilih hanya dari daftar ketinggian tersebut.
select_parents(self, population, fitness_scores)	Memilih dua orang tua dengan *roulette wheel selection* dengan pengembalian, dengan peluang sebanding *fitness*. Jika seluruh *fitness* bernilai nol, pemilihan dilakukan seragam.
_empty_state(self)	Membuat *state* kosong, yaitu semua paket tidak dimuat.

Dilanjutkan ke halaman berikutnya

Tabel 3.12 – lanjutan

Kelas/Fungsi	**Deskripsi**
_can_insert(self, pid, position, orientation, inside)	Mengecek apakah sebuah paket dapat disisipkan pada posisi dan orientasi tertentu terhadap paket yang sudah berada di dalam truk (inside), mencakup batas truk, *overlap*, paket *fragile*, dan penyangga.
_insert(self, state, pid, orientation, inside, loaded_weight, rng, preferred)	Mencoba menyisipkan paket ke *state*. Pengecekan berat dilakukan lebih dahulu. Posisi preferred (jika ada) dicoba lebih dahulu, kemudian hingga REPAIR_ATTEMPTS posisi acak. Mengembalikan True bila berhasil.
_generate_initial_state(self, rng)	Membangkitkan satu individu acak dengan menyisipkan paket dalam urutan acak.
crossover(self, parent_a, parent_b)	Menghasilkan satu anak dari dua orang tua melalui pewarisan gen per paket.
mutate(self, state)	Menerapkan satu *mutation* (*swap*, *move*, atau *rotate*) yang menghasilkan *state* valid; jika gagal, mengembalikan salinan *state* semula.
_statistics(iteration, scores, best_value)	Menghitung statistik populasi per generasi (nilai maksimum, rata-rata, dan nilai terbaik sejauh ini).
genetic_algorithm(self, population_size=50, iterations=100)	Menjalankan seluruh proses evolusi dan mengembalikan hasil dalam format build_result.

Kode 3.21 menunjukkan konstanta dan konstruktor, serta fungsi pembangkit posisi acak di dalam truk dan seleksi orang tua. Seleksi menggunakan random.choices dengan bobot berupa nilai *fitness* yang sama dengan nilai *objective function*. Seleksi dilakukan dengan pengembalian sehingga kedua orang tua dapat merupakan individu yang sama, dan akan muncul ValueError bila terdapat *fitness* negatif.

```
1 import random
2 import time
3
```

```python
from src.algorithms.common import build_result
from src.constraints import validate_state
from src.objective import evaluate
from src.package import ORIENTATIONS
from src.placement import Placement
from src.state import State


class GeneticAlgorithm:
    CROSSOVER_PROBABILITY = 0.80
    MUTATION_PROBABILITY = 0.30
    REPAIR_ATTEMPTS = 20
    MUTATION_ATTEMPTS = 10

    def __init__(self, problem, rng):
        self.problem = problem
        self.rng = rng
        self.packages = problem.package_map
```

Kode 3.21: Konstanta dan konstruktor GeneticAlgorithm

```python
    def print(self):
        print(f"GeneticAlgorithm(packages={len(self.problem.packages)})")

    def random_inside_position(self, package, orientation, rng=None, heights=None):
        rng = self.rng if rng is None else rng
        dx, dy, dz = package.effective_dims(orientation)
        limits = (self.problem.truck.w - dx, self.problem.truck.l - dy,self.problem.truck.h - dz)

        if min(limits) < 0:
            return None

        if heights is None:
            z = rng.randint(0, limits[2])
        else:
            allowed = [height for height in heights if 0 <= height <= limits[2]]
            if not allowed:
```

```
                return None
            z = rng.choice(allowed)

        return [rng.randint(0, limits[0]), rng.randint(0, limits[1]), z]

    def select_parents(self, population, fitness_scores):
        if any(score < 0 for score in fitness_scores):
            raise ValueError("Fitness scores must be nonnegative.")
```

Kode 3.22: Pembangkit posisi acak dan seleksi orang tua

Metode _can_insert dan _insert pada Kode 3.23 merupakan versi inkremental dari pengecekan *constraint*. _can_insert menolak posisi yang keluar dari truk, bertumpang tindih dengan paket lain, berada di atas paket *fragile*, atau (bila paket yang disisipkan *fragile*) berada di bawah paket lain, serta mensyaratkan paket berada di lantai atau tepat di atas paket *non-fragile*. _insert lebih dahulu menolak paket yang membuat total berat melebihi kapasitas, kemudian menyusun daftar ketinggian kandidat, yaitu lantai dan sisi atas paket *non-fragile* yang sudah ada, dan mencoba posisi acak pada ketinggian tersebut hingga REPAIR_ATTEMPTS kali. Paket yang gagal disisipkan dibiarkan berada di luar truk.

```
        weights = fitness_scores if sum(fitness_scores) else None
        return tuple(self.rng.choices(population, weights=weights, k=2))

    def _empty_state(self):
        return State({
            pid: Placement(None, ["x", "y", "z"])
            for pid in self.packages
        })

    def _can_insert(self, pid, position, orientation, inside):
        package = self.packages[pid]
        truck = self.problem.truck
        x, y, z = position
        dx, dy, dz = package.effective_dims(orientation)

        if not (0 <= x <= truck.w - dx and 0 <= y <= truck.l - dy and 0 <=
z <= truck.h - dz):
            return False

```

```
        supported = z == 0
        for other_id, (ox, oy, oz, odx, ody, odz) in inside.items():
            if not (x < ox + odx and ox < x + dx and y < oy + ody and oy < y + dy):
                continue

            if z < oz + odz and oz < z + dz:
                return False
            if self.packages[other_id].is_fragile and z >= oz + odz:
                return False
            if package.is_fragile and oz >= z + dz:
                return False
            if oz + odz == z and not self.packages[other_id].is_fragile:
                supported = True

        return supported

    def _insert(self, state, pid, orientation, inside, loaded_weight, rng, preferred=None):
        package = self.packages[pid]
        if loaded_weight + package.weight > self.problem.truck.max_capacity:
            return False

        heights = sorted({0} | {box[2] + box[5]
                for other_id, box in inside.items()
                if not self.packages[other_id].is_fragile
            }
        )
        attempts = self.REPAIR_ATTEMPTS + (preferred is not None)

        for attempt in range(attempts):
            if attempt == 0 and preferred is not None:
                position = list(preferred)
            else:
                position = self.random_inside_position(package, orientation, rng=rng, heights=heights)

            if position is None:
                return False
```

```
55
56              if self._can_insert(pid, position, orientation, inside):
57                  state.placements[pid] = Placement(position, list(
orientation))
58                  inside[pid] = (*position, *package.effective_dims(
orientation))
```

Kode 3.23: Pengecekan dan penyisipan paket secara inkremental

Individu populasi awal dibangkitkan oleh _generate_initial_state pada Kode 3.24. Urutan paket diacak, orientasi unik (yang menghasilkan dimensi efektif berbeda) diacak, dan setiap paket dicoba disisipkan dengan salah satu orientasi hingga berhasil.

```
1
2           return False
3
4       def _generate_initial_state(self, rng):
5           state = self._empty_state()
6           inside, loaded_weight = {}, 0
7           ids = list(self.packages)
8           rng.shuffle(ids)
9
10          for pid in ids:
11              orientations = list({
12                  self.packages[pid].effective_dims(orientation): orientation
13                  for orientation in ORIENTATIONS
14              }.values())
15              rng.shuffle(orientations)
16              for orientation in orientations:
17                  if self._insert(state, pid, orientation, inside,
loaded_weight, rng):
18                      loaded_weight += self.packages[pid].weight
```

Kode 3.24: Pembangkitan individu awal

Operator *crossover* pada Kode 3.25 bekerja per paket (*uniform crossover*): untuk setiap paket, gen (posisi dan orientasi) diwarisi dari salah satu orang tua yang dipilih acak. Anak dibangun dengan menyisipkan kembali paket-paket berposisi tidak None secara berurutan dari ketinggian z terkecil, dengan posisi warisan sebagai percobaan pertama dan posisi acak sebagai perbaikan bila posisi warisan tidak valid. Dengan

cara ini, anak selalu valid, tetapi sebagian paket dapat tidak termuat.

```
        return state

    def crossover(self, parent_a, parent_b):
        inherited = {
            pid: self.rng.choice((parent_a, parent_b)).placements[pid]
            for pid in self.packages
        }
        child = State({
            pid: Placement(None, list(gene.orientation))
            for pid, gene in inherited.items()
        })
        ids = [pid for pid, gene in inherited.items() if gene.position is
not None]
        self.rng.shuffle(ids)
        ids.sort(key=lambda pid: inherited[pid].position[2])
        inside, loaded_weight = {}, 0

        for pid in ids:
            gene = inherited[pid]
            if self._insert(
                child, pid, gene.orientation, inside, loaded_weight,
                self.rng, preferred=gene.position,
            ):
```

Kode 3.25: Operator *crossover*

Operator *mutation* pada Kode 3.26 memilih secara seragam salah satu dari tiga jenis perubahan: *swap* (menukar posisi dua paket), *move* (memindahkan satu paket ke posisi acak, dengan peluang 20% paket dikeluarkan dari truk), atau *rotate* (merotasi satu paket pada sumbu acak, hanya dihitung bila dimensi efektif berubah). Kandidat harus berbeda dari *state* semula dan lolos validate_state. Hingga MUTATION_ATTEMPTS percobaan dilakukan, dan bila tidak ada yang valid, salinan *state* semula dikembalikan.

```
        return child

```

```
    def mutate(self, state):
        ids = list(state.placements)
        if not ids:
            return state.copy()

        operation = self.rng.choice(("swap", "move", "rotate"))
        original_key = state.key()

        for _ in range(self.MUTATION_ATTEMPTS):
            candidate = state.copy()

            if operation == "swap":
                if len(ids) < 2:
                    break
                first_id, second_id = self.rng.sample(ids, 2)
                first = candidate.placements[first_id]
                second = candidate.placements[second_id]
                first.position, second.position = second.position, first.
position

            elif operation == "move":
                pid = self.rng.choice(ids)
                placement = candidate.placements[pid]
                if self.rng.random() < 0.20:
                    placement.position = None
                else:
                    position = self.random_inside_position(
                        self.packages[pid], placement.orientation
                    )
                    if position is None:
                        continue
                    placement.position = position

            else:
                pid = self.rng.choice(ids)
                placement = candidate.placements[pid]
                before = self.packages[pid].effective_dims(placement.
orientation)
                axis = self.rng.choice(("x", "y", "z"))
                placement.rotate(axis)
```

```
                if self.packages[pid].effective_dims(placement.orientation)
== before:
                    continue

            if candidate.key() == original_key:
                continue

            valid, _ = validate_state(self.problem, candidate)
            if valid:
```

Kode 3.26: Operator *mutation*

Proses evolusi pada Kode 3.27 berjalan sebagai berikut. Populasi awal dibangkitkan dengan generator acak terpisah yang diturunkan dari rng, kemudian dievaluasi, dan individu terbaiknya dicatat sebagai initial_state. Pada setiap generasi, populasi baru dibentuk dengan mengulangi langkah berikut hingga ukurannya sama dengan population_size: dua orang tua dipilih dengan *roulette wheel*; dengan probabilitas 0,80 anak dibentuk melalui *crossover*, selain itu anak berupa salinan salah satu orang tua; kemudian dengan probabilitas 0,30 anak dimutasi. Populasi baru menggantikan seluruh populasi lama (tanpa *elitism*), tetapi individu terbaik sepanjang pencarian disimpan terpisah pada best_state. Pencarian berhenti setelah iterations generasi dengan alasan iteration_limit_reached, dan jumlah evaluasi *objective function* adalah population_size $\times$ (iterations $+1$). Pada main.py, eksperimen dijalankan pada dua rangkaian: populasi sebagai kontrol dengan tiga variasi jumlah iterasi, dan iterasi sebagai kontrol dengan tiga variasi ukuran populasi.

```

        return state.copy()

    @staticmethod
    def _statistics(iteration, scores, best_value):
        return {
            "iteration": iteration,
            "population_max": max(scores),
            "population_mean": sum(scores) / len(scores),
            "best_so_far": best_value,
        }

    def genetic_algorithm(self, population_size=50, iterations=100):
```

```
        if population_size < 2:
            raise ValueError("Population size must be at least 2.")
        if iterations < 1:
            raise ValueError("Iterations must be positive.")

        started = time.perf_counter()
        initialization_rng = random.Random(self.rng.getrandbits(128))
        population = [
            self._generate_initial_state(initialization_rng)
            for _ in range(population_size)
        ]
        initial_population = population
        scores = [evaluate(self.problem, state) for state in population]
        best_index = max(range(population_size), key=scores.__getitem__)
        best_state = population[best_index].copy()
        best_value = scores[best_index]
        initial_state, initial_value = best_state.copy(), best_value
        history = [self._statistics(0, scores, best_value)]
        best_generation = crossovers = mutations = accepted_mutations = 0

        for generation in range(1, iterations + 1):
            next_population = []

            while len(next_population) < population_size:
                parent_a, parent_b = self.select_parents(population, scores
)

                if self.rng.random() < self.CROSSOVER_PROBABILITY:
                    child = self.crossover(parent_a, parent_b)
                    crossovers += 1
                else:
                    child = self.rng.choice((parent_a, parent_b)).copy()

                if self.rng.random() < self.MUTATION_PROBABILITY:
                    mutations += 1
                    before = child.key()
                    child = self.mutate(child)
                    accepted_mutations += child.key() != before

                next_population.append(child)
```

```
53
54              population = next_population
55              scores = [evaluate(self.problem, state) for state in population
    ]
56              best_index = max(range(population_size), key=scores.__getitem__
    )
57
58              if scores[best_index] > best_value:
59                  best_state = population[best_index].copy()
60                  best_value = scores[best_index]
61                  best_generation = generation
```

Kode 3.27: Statistik populasi dan proses evolusi *Genetic Algorithm*

```
1
2           duration = time.perf_counter() - started
3
4           return build_result(
5               algorithm="genetic_algorithm",
6               parameters={"population_size": population_size, "iterations":
    iterations},
7               initial_state=initial_state,
8               initial_value=initial_value,
9               final_state=best_state,
10              final_value=best_value,
11              iterations=iterations,
12              accepted_moves=accepted_mutations,
13              objective_evaluations=population_size * (iterations + 1),
14              duration_seconds=duration,
15              stop_reason="iteration_limit_reached",
16              history=history,
17              extra={
18                  "initial_state_selection": "
    best_individual_in_initial_population",
19                  "initial_population": [state.to_dict() for state in
    initial_population],
20                  "final_population_fitness": scores,
21                  "best_generation": best_generation,
22                  "crossovers": crossovers,
23                  "mutation_calls": mutations,
```

```
                "accepted_mutations": accepted_mutations,
                "settings": {
                    "selection": "roulette_wheel_with_replacement",
                    "crossover_probability": self.CROSSOVER_PROBABILITY,
                    "mutation_probability": self.MUTATION_PROBABILITY,
                    "elitism": False,
                    "repair_attempts": self.REPAIR_ATTEMPTS,
                    "mutation_attempts": self.MUTATION_ATTEMPTS,
```

Kode 3.28: Penyusunan hasil *Genetic Algorithm*

Bab 4

Hasil Eksperimen dan Analisis

4.1 Eksperimen Algoritma *Hill-climbing*

4.1.1 *Steepest Ascent Hill-climbing*

4.1.2 *Hill-climbing with Sideways Move*

4.1.3 *Random Restart Hill-climbing*

4.1.4 *Stochastic Hill-climbing*

4.2 Eksperimen Algoritma *Simulated Annealing*

4.3 Eksperimen Algoritma *Genetic Algorithm*

4.4 Analisis Hasil Eksperimen

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

Referensi

Institut Teknologi Bandung. (2026). *Beyond classical search* [Slide kuliah IF3170 Inteligensi Artifisial, Semester I TA 2026/2027]. Program Studi Teknik Informatika, Institut Teknologi Bandung.

Lampiran

Pembagian Tugas

Tabel 1: Pembagian tugas anggota kelompok

No	Nama Anggota	Tugas
1	Nathan E C Marpaung	• AAAA
2	Kurt Mikhael Purba	• AAAAA
3	Aurelia Jennifer Gunawan	• AAAAA

Form Pengunaan AI